



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных Технологий

Кафедра Вычислительной техники

**ОТЧЕТ О ВЫПОЛНЕНИИ ПРАКТИЧЕСКОЙ РАБОТЫ
№6**

«Реализация синтаксического анализатора (с минимум одной
семантической проверкой) на Python»

**по дисциплине
«Теория формальных языков»**

Выполнил студент группы ИКБО-43-23

Кошечев М. И.

Принял ассистент

Цынгальев П.С.

Практическая работа
выполнена

«15» декабря 2024 г.

«Зачтено»

«__» _____ 2024 г.

Москва 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ЦЕЛЬ	4
2 ЗАДАНИЕ	5
2.1 Формулировка задания	5
2.2 Математическая модель.....	6
2.3 Тестирование программы	7
3 ЗАКЛЮЧЕНИЕ	12
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	13
ПРИЛОЖЕНИЯ.....	14

ВВЕДЕНИЕ

Изучение теории формальных языков представляет собой значимый элемент образовательной программы по направлению программной инженерии. Эта область, являющаяся ключевой частью компьютерных наук и теоретической информатики, сосредотачивается на исследовании структуры и свойств формальных языков, а также на моделировании их обработки. В условиях современного общества, где возрастают объемы данных и потребность в эффективных методах их хранения, передачи и обработки, понимание основ формальных языков приобретает особую значимость.

Формальные языки находят применение в различных сферах: от описания синтаксиса языков программирования до разработки сетевых протоколов и создания компиляторов. С их помощью удастся формализовать взаимодействие как между человеком и компьютером, так и между компонентами программного обеспечения. В центре изучения теории формальных языков находятся такие понятия, как грамматики, автоматы и алгебраические структуры, которые позволяют описывать синтаксис и семантику языков.

В данном отчете мы рассмотрим программу реализацию синтаксического анализатора (с минимум одной семантической проверкой) на языке Python.

1 ЦЕЛЬ

Реализовать синтаксический анализатор (с минимум одной семантической проверкой) на языке Python, согласно варианту КР.

2 ЗАДАНИЕ

2.1 Формулировка задания

На выбранном ЯП реализовать синтаксический анализатор (с минимум одной семантической проверкой), согласно варианту КР

Согласно индивидуальному варианту задания на курсовую работу грамматика языка включает следующие синтаксические конструкции:

`<операции_группы_отношения> ::= <> | = | < | <= | > | >=`

`<операции_группы_сложения> ::= + | - | or`

`<операции_группы_умножения> ::= * | / | and`

`<унарная_операция> ::= not`

`<программа> ::= program var <описание> begin <оператор> { ; <оператор> } end.`

`<описание> ::= { <идентификатор> { , <идентификатор> } : <тип> ; }`

`<тип> ::= % | ! | $`

`<оператор> ::= <составной> | <присваивания> | <условный> | <фиксированного_цикла> | <условного_цикла> | <ввода> | <вывода>`

`<составной> ::= «[» <оператор> { ( : | перевод строки) <оператор> } «]»`

`<присваивания> ::= <идентификатор> as <выражение>`

`<условный> ::= if <выражение> then <оператор> [ else <оператор> ]`

`<фиксированного_цикла> ::= for <присваивания> to <выражение> do <оператор>`

`<условного_цикла> ::= while <выражение> do <оператор>`

`<ввода> ::= read «(» <идентификатор> { , <идентификатор> } «)»`

`<вывода> ::= write «(» <выражение> { , <выражение> } «)»`

```

{ ... }

<выражение> ::=      <операнд> { <операции_группы_отношения>
<операнд> }

<операнд> ::=      <слагаемое>      { <операции_группы_сложения>
<слагаемое> }

<слагаемое> ::= <множитель> { <операции_группы_умножения>
<множитель> }

<множитель> ::=      <идентификатор>          |      <число>
| <логическая_константа> |

<унарная_операция> <множитель> | «(» <выражение> «)»

<число> ::= <целое> | <действительное>

<логическая_константа> ::= true | false

<идентификатор> ::= <буква> { <буква> | <цифра> }

<буква> ::= A | B | C | D | E | F | G | H | I | J | K |
L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z |
a | b | c | d | e | f | g | h | i | j | k | l | m | n | o |
p | q | r | s | t | u | v | w | x | y | z

<цифра> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

```

2.2 Математическая модель

1. Синтаксический анализатор получает список лексем. Основная роль синтаксического анализатора заключается в управлении процессом разбора, который организуется в виде набора взаимосвязанных методов, каждый из которых соответствует определенному правилу грамматики. Представленный код реализует синтаксический анализатор с использованием метода рекурсивного спуска. Этот подход базируется на ассоциации рекурсивных функций с нетерминалами грамматики. Например, для разбора программы используется метод `parse_program`, который обрабатывает ключевые компоненты грамматики, такие как определения переменных, блоки операторов и завершение программы.

Исходный код синтаксического анализатора приведен в Приложении А.

2. Семантический анализатор

Некоторые особенности модельного языка не могут быть выражены с помощью контекстно-свободной грамматики. К таким особенностям относятся следующие правила:

1. Каждый идентификатор, используемый в программе, должен быть предварительно описан.
2. Повторное объявление одного и того же идентификатора запрещено.
3. В операторе присваивания типы переменных должны быть одинаковыми.
4. Условные операторы и операторы циклов могут использовать только логические выражения в качестве условий.
5. Операнды отношений должны быть целочисленными.
6. Эти правила анализируются на этапе семантического анализа.
7. Декларация переменных.
8. При присваивании проверяется, что переменная объявлена и что тип переменной совпадает с присваиваемым значением.
9. Для условных операторов условие является логическим выражением.
10. Обработка различных типов чисел, а также проверка совместимости типов. Логические выражения проверяются на соответствие типу \$.

Описания функций семантических проверок приведены в листинге в Приложении Б.

2.3 Тестирование программы

Разработано консольное приложение, предназначенное для анализа исходного текста программы, написанной на модельном языке. Программа делает синтаксический и семантический анализ введенного кода. В результате выполнения приложение либо подтверждает правильность программы, либо выводит сообщение об ошибке. Рассмотрим несколько примеров работы

приложения. Рассмотрим несколько примеров работы приложения. Исходный код программы приведен в Листинге 1.

Листинг 1 – main.py

```
program
var x, y : integer;
begin
    x := 5; { Присваиваем x значение 5 }
    y := 10; { Присваиваем y значение 10 }
end.
```



```
* Синтаксический анализ *
Текущий токен: ('KEYWORD', 'program'), следующий токен: ('KEYWORD', 'var')
Начинаем синтаксический анализ...
Начинаем разбор программы, текущий токен: ('KEYWORD', 'program')
Текущий токен: ('KEYWORD', 'var'), следующий токен: ('ID', 'x')
Проверка токена в блоке: ('KEYWORD', 'var')
Начинаем разбор объявлений переменных.
Текущий токен: ('ID', 'x'), следующий токен: ('DELIMITER', ',')
Текущий токен: ('DELIMITER', ','), следующий токен: ('ID', 'y')
Текущий токен: ('ID', 'y'), следующий токен: ('DELIMITER', ':')
Текущий токен: ('DELIMITER', ':'), следующий токен: ('KEYWORD', 'integer')
Текущий токен: ('KEYWORD', 'integer'), следующий токен: ('DELIMITER', ';')
Текущий токен: ('DELIMITER', ';'), следующий токен: ('KEYWORD', 'begin')
Текущий токен: ('KEYWORD', 'begin'), следующий токен: ('ID', 'x')
Текущий токен: ('ID', 'x'), следующий токен: ('ASSIGN', ':=')
Начинаем разбор операторов.
Обрабатываем оператор присваивания.
Обрабатываем оператор присваивания: x
Текущий токен: ('ASSIGN', ':='), следующий токен: ('NUMBER', '5')
Текущий токен: ('NUMBER', '5'), следующий токен: ('DELIMITER', ';')
Обрабатываем выражение с текущим токеном: ('NUMBER', '5')
Обрабатываем терм с текущим токеном: ('NUMBER', '5')
Обрабатываем фактор с текущим токеном: ('NUMBER', '5')
Текущий токен: ('DELIMITER', ';'), следующий токен: ('ID', 'y')
Текущий токен: ('ID', 'y'), следующий токен: ('ASSIGN', ':=')
Обрабатываем оператор присваивания.
Обрабатываем оператор присваивания: y
Текущий токен: ('ASSIGN', ':='), следующий токен: ('NUMBER', '10')
Текущий токен: ('NUMBER', '10'), следующий токен: ('DELIMITER', ';')
Обрабатываем выражение с текущим токеном: ('NUMBER', '10')
Обрабатываем терм с текущим токеном: ('NUMBER', '10')
Обрабатываем фактор с текущим токеном: ('NUMBER', '10')
```

Рисунок 1.1 - Тестирование синтаксического и семантического анализа

```
Текущий токен: ('DELIMITER', ';'), следующий токен: ('KEYWORD', 'end')
Текущий токен: ('KEYWORD', 'end'), следующий токен: ('DELIMITER', '.')
Текущий токен перед 'end': ('KEYWORD', 'end')
Текущий токен: ('DELIMITER', '.'), следующий токен: None
Токены закончились.
Синтаксический анализ завершен. Статус: OK
```

Рисунок 1.2 - Тестирование синтаксического и семантического анализа

Исходный код программы, содержащий синтаксическую ошибку, приведен в листинге 2.

Листинг 2 – main.py

```
program
var x, y : integer;
begin
    1x := 5; { Присваиваем x значение 5 }
    y := 10; { Присваиваем y значение 10 }
end.
```

* Синтаксический анализ *

```
Текущий токен: ('KEYWORD', 'program'), следующий токен: ('KEYWORD', 'var')
Начинаем синтаксический анализ...
Начинаем разбор программы, текущий токен: ('KEYWORD', 'program')
Текущий токен: ('KEYWORD', 'var'), следующий токен: ('ID', 'x')
Проверка токена в блоке: ('KEYWORD', 'var')
Начинаем разбор объявлений переменных.
Текущий токен: ('ID', 'x'), следующий токен: ('DELIMITER', ',')
Текущий токен: ('DELIMITER', ','), следующий токен: ('ID', 'y')
Текущий токен: ('ID', 'y'), следующий токен: ('DELIMITER', ':')
Текущий токен: ('DELIMITER', ':'), следующий токен: ('KEYWORD', 'integer')
Текущий токен: ('KEYWORD', 'integer'), следующий токен: ('DELIMITER', ';')
Текущий токен: ('DELIMITER', ';'), следующий токен: ('KEYWORD', 'begin')
Текущий токен: ('KEYWORD', 'begin'), следующий токен: ('NUMBER', '1')
Текущий токен: ('NUMBER', '1'), следующий токен: ('ID', 'x')
Ошибка синтаксического анализа: Неожиданный оператор: 1x
```

Рисунок 2 - Тестирование ошибки синтаксического анализа

Исходный код программы, содержащий семантическую ошибку, приведен в листинге 3.

Листинг 3 – main.py

```
program
var x, y : integer;
begin
    d := 5; { Присваиваем x значение 5 }
    y := 10; { Присваиваем y значение 10 }
end.
```

```
Проверяем переменную для присваивания: y
Текущие ошибки: ["Ошибка: Переменная 'd' не объявлена.", "Ошибка: Переменная 'd' не объявлена."]
Семантический анализ завершен.
Обнаружены ошибки семантического анализа:
Ошибка: Переменная 'd' не объявлена.
```

Рисунок 3 - Тестирование ошибки семантического анализа

3 ЗАКЛЮЧЕНИЕ

В процессе выполнения практической работы были достигнуты следующие результаты:

Разбором исходного текста программы занимается синтаксический анализатор, который реализован в виде класса `SyntaxAnalyzer` на языке Python. Анализатор распознает входной язык по методу рекурсивного спуска. Для применимости необходимо было преобразовать грамматику, в частности, специальным образом обрабатывать встречающиеся итеративные синтаксически конструкции.

В код рекурсивных функций включены проверки дополнительных семантических условий, в частности, проверка на повторное объявление одной и той же переменной.

Тестирование программного продукта показало, что синтаксически и семантически корректно написанная программа успешно распознается анализатором, а программа, содержащая ошибки, отвергается.

Таким образом, поставленная цель работы — Реализовать синтаксический анализатор (с минимум одной семантической проверкой) на языке Python, согласно варианту КР — успешно достигнута.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Малявко А.А. Формальные языки и компиляторы: учебное пособие для вузов. - М., Изд-во Юрайт, 2019.
2. Свердлов С.З. Языки программирования и методы трансляции : учебное пособие для вузов - СПб., 2007, Id=65534.
3. Карпов Ю.Г. Теория и технология программирования. Основы построения трансляторов: учеб. пособие. - СПб.: БХВ-Петербург, 2005, Id=64347.

ПРИЛОЖЕНИЯ

Приложение А – Класс синтаксического анализатора

Приложение Б – Класс семантического анализатора

ПРИЛОЖЕНИЕ А

Класс синтаксического анализатора

Листинг А.1 – *parserr.py*

```
class SyntaxAnalyzer:
def __init__(self, tokens):
    self.tokens = deque(tokens)
    self.current_token = None
    self.next_token()

def next_token(self):
    if self.tokens:
        self.current_token = self.tokens.popleft()
        next_token = self.tokens[0] if self.tokens else None
        print(f"Текущий token: {self.current_token},  
следующий token: {next_token}")
        if self.current_token[0]=="NUMBER" and  
next_token[0]=="ID":
            raise SyntaxError(f"Неожиданный оператор:  
{self.current_token[1]}{next_token[1]}")

        else:
            self.current_token = None
            print("Токены закончились.")

def parse(self):
    print("Начинаем синтаксический анализ...")
    self.program()
    if self.current_token is not None:
        raise SyntaxError(f"Неожиданный token:  
{self.current_token}")
    return "OK"

def program(self):
    print(f"Начинаем разбор программы, текущий token:  
{self.current_token}")
    if self.current_token[0] == 'KEYWORD' and  
self.current_token[1] == 'program':
        self.next_token()
        self.block()
    else:
        raise SyntaxError("Ожидалось 'program'.")
```

```

        def block(self):
            print(f"Проверка токена в блоке: {self.current_token}")
            if self.current_token[0] == 'KEYWORD' and
self.current_token[1] == 'var':
                self.variable_declarations()
            found_begin = False
            while self.current_token is not None:
                if self.current_token[0] == 'KEYWORD' and
self.current_token[1] == 'begin':
                    found_begin = True
                    self.next_token()
                    break
                self.next_token()
            if not found_begin:
                raise SyntaxError("Ожидалось 'begin' после объявления
переменных.")
            self.statements()
            print(f"Текущий токен перед 'end': {self.current_token}")
            if self.current_token[0] == 'KEYWORD' and
self.current_token[1] == 'end':
                self.next_token()
                if self.current_token[0] == 'DELIMITER' and
self.current_token[1] == '.':
                    self.next_token()
                else:
                    raise SyntaxError("Ожидалась точка '.' после
'end'.")
            else:
                raise SyntaxError("Ожидалось 'end' после блока.")

        def variable_declarations(self):
            print(f"Начинаем разбор объявлений переменных.")
            while self.current_token[0] == 'ID':
                print(f"Обрабатываем переменную:
{self.current_token[1]}")
                self.next_token()
                if self.current_token[0] == 'DELIMITER' and
self.current_token[1] == ',':
                    self.next_token()
                elif self.current_token[0] == 'DELIMITER' and
self.current_token[1] == ';':
                    self.next_token()
                    break
            else:
                raise SyntaxError("Ожидалось ',' или ';' после
объявления переменной.")

```



```

def statements(self):
    print(f"Начинаем разбор операторов.")
    while self.current_token is not None and
    (self.current_token[0] != 'KEYWORD' or self.current_token[1] !=
    'end'):
        if self.current_token[0] == 'ID' and self.tokens and
self.tokens[0][1] == ':=':
            print("Обрабатываем оператор присваивания.")
            self.assignment_statement()
        elif self.current_token[0] == 'KEYWORD' and
self.current_token[1] == 'if':
            print("Обрабатываем условие 'if'.")
            self.if_statement()
        elif self.current_token[0] == 'KEYWORD' and
self.current_token[1] == 'while':
            print("Обрабатываем условие 'while'.")
            self.while_statement()
        else:
            raise SyntaxError(f"Неожиданный оператор:
{self.current_token}")

def assignment_statement(self):
    print(f"Обрабатываем оператор присваивания:
{self.current_token[1]}")
    var_name = self.current_token[1]
    self.next_token()
    if self.current_token[0] == 'ASSIGN' and
self.current_token[1] == ':=':
        self.next_token()
        self.expression()
    if self.current_token[0] == 'DELIMITER' and
self.current_token[1] == ';':
        self.next_token()
    else:
        raise SyntaxError("Ожидался символ ';' после
присваивания.")
    else:
        raise SyntaxError("Ожидалось ':' в операторе
присваивания.")

```

```

        def expression(self):
            print(f"Обрабатываем выражение с текущим токеном:
{self.current_token}")
            self.term()
            while self.current_token is not None and
self.current_token[0] in ['ADD_OP', 'SUB_OP']:
                print(f"Обрабатываем операцию:
{self.current_token[1]}")
                self.next_token()
                self.term()

        def term(self):
            print(f"Обрабатываем терм с текущим токеном:
{self.current_token}")
            self.factor()
            while self.current_token is not None and
self.current_token[0] in ['MUL_OP', 'DIV_OP']:
                print(f"Обрабатываем операцию:
{self.current_token[1]}")
                self.next_token()
                self.factor()

        def factor(self):
            print(f"Обрабатываем фактор с текущим токеном:
{self.current_token}")
            if self.current_token[0] == 'ID' or self.current_token[0] ==
'NUMBER':
                self.next_token()
            else:
                raise SyntaxError(f"Неожиданный токен в факторе:
{self.current_token}")

        def write_statement(self):
            print(f"Обрабатываем оператор write с текущим токеном:
{self.current_token}")
            self.next_token()
            if self.current_token[0] == 'DELIMITER' and
self.current_token[1] == '(':
                self.next_token()
                self.expression()
                if self.current_token[0] == 'DELIMITER' and
self.current_token[1] == ')':
                    self.next_token()
                else:
                    raise SyntaxError("Ожидалась закрывающая скобка ')'
после аргумента write.")
            else:
                raise SyntaxError("Ожидалась открывающая скобка '('
после write.")

```

```

def if_statement(self):
    print(f"Обрабатываем оператор 'if' с текущим токеном:
{self.current_token}")
    self.next_token()
    self.expression()
    if self.current_token[0] == 'REL_OP' and
self.current_token[1] in ['>', '<', '=', '>=', '<=']:
        self.next_token()
        self.next_token()
        if self.current_token[0] == 'KEYWORD' and
self.current_token[1] == 'then':
            self.next_token()
            if self.current_token[0] == 'DELIMITER' and
self.current_token[1] == '[':
                self.next_token()
                while self.current_token[0] != 'DELIMITER' or
self.current_token[1] != ']':
                    if self.current_token[0] == 'KEYWORD' and
self.current_token[1] == 'write':
                        self.write_statement()
                    elif self.current_token[0] == 'DELIMITER'
and self.current_token[1] == ';':
                        self.next_token()
                    else:
                        self.statements()
                if self.current_token[0] == 'DELIMITER' and
self.current_token[1] == ']':
                    self.next_token()
                else:
                    raise SyntaxError("Ожидался закрывающий ']'
после блока операторов.")
            else:
                raise SyntaxError("Ожидался блок операторов
после 'then'.")
        if self.current_token[0] == 'KEYWORD' and
self.current_token[1] == 'else':
            self.next_token()
            if self.current_token[0] == 'DELIMITER' and
self.current_token[1] == '[':
                self.next_token()
                while self.current_token[0] != 'DELIMITER'
or self.current_token[1] != ']':
                    if self.current_token[0] == 'KEYWORD'
and self.current_token[1] == 'write':
                        self.write_statement()
                    elif self.current_token[0] ==
'DELIMITER' and self.current_token[1] == ';':
                        self.next_token()

```

```
        else:
            self.statements()
            if self.current_token[0] == 'DELIMITER' and
self.current_token[1] == ']':
                self.next_token()
            else:
                raise SyntaxError("Ожидался закрывающий
']' после блока операторов в 'else'.")
            else:
                self.statements()
        else:
            raise SyntaxError("Ожидалось 'then' после условия
'if'.")
        else:
            raise SyntaxError("Ожидался оператор сравнения после
условия 'if'.")

def while_statement(self):
    print(f"Обрабатываем оператор 'while' с текущим токеном:
{self.current_token}")
    self.next_token()
    self.expression()
    if self.current_token[0] == 'KEYWORD' and
self.current_token[1] == 'do':
        self.next_token()
        self.statements()
    else:
        raise SyntaxError("Ожидалось 'do' после условия
'while'.")
```

ПРИЛОЖЕНИЕ Б

Класс синтаксического анализатора

Листинг Б.1 – semantic.py

```
class SemanticAnalyzer:
def __init__(self, symbol_table):
    self.symbol_table = symbol_table
    self.errors = []

def analyze(self, operations):
    print("Начало семантического анализа...")
    for operation in operations:
        print(f"Обрабатываем операцию: {operation}")
        if operation[0] == 'assign':
            variable = operation[1]
            print(f"Проверяем переменную для присваивания: {variable}")
            if variable not in self.symbol_table:
                self.errors.append(f"Ошибка: Переменная '{variable}' не объявлена.")
            elif operation[0] == 'use':
                variable = operation[1]
                print(f"Проверяем использование переменной: {variable}")
                if variable not in self.symbol_table:
                    self.errors.append(f"Ошибка: Переменная '{variable}' не объявлена.")
            else:
                self.errors.append(f"Ошибка: Неизвестная операция '{operation[0]}'.")
        print(f"Текущие ошибки: {self.errors}")
    print("Семантический анализ завершен.")
    return self.errors

def get_errors(self):
    return self.errors
```

```

def generate_symbol_table_and_operations(tokens):
global
symbol_table = {}
operations = []
current_type = None
in_var_section = False
var_seen = False
begin_seen = False

for j, token in enumerate(tokens):
    print(f"Обрабатываем токен: {token}")
    if token[0] == 'KEYWORD':
        if token[1] in ['%', '!', '$']:
            current_type = token[1]
            global_type=token[1]
            print(current_type)

for i, token in enumerate(tokens):
    print(f"Обрабатываем токен: {token}")
    if token[0] == 'KEYWORD':
        if token[1] == 'var':
            var_seen = True
            in_var_section = True
        elif token[1] in ['integer', 'real', 'boolean'] and
var_seen and not begin_seen:
            current_type = token[1]
        elif token[1] == 'begin':
            begin_seen = True
            in_var_section = False
            current_type = None
        elif token[0] == 'ID' and var_seen and not begin_seen:
            if token[1] not in symbol_table:
                symbol_table[token[1]] = {'type':
current_type, 'scope': 'global'}
                print(global_type)
                print(f"Переменная '{token[1]}' добавлена в
таблицу символов с типом '{current_type}'")
            elif token[0] == 'ID' and not in_var_section:
                if token[1] not in symbol_table:
                    operations.append(('use', token[1]))
                    print(f"Переменная '{token[1]}' используется,
но не найдена в таблице символов.")
                else:
                    operations.append(('use', token[1]))
                    print(f"Переменная '{token[1]}' используется и
найденa в таблице символов.")

```

Продолжение листинга Б.1

```
elif token[0] == 'ASSIGN':
    if i > 0 and tokens[i - 1][0] == 'ID':
        var_name = tokens[i - 1][1]
        if var_name not in symbol_table:
            operations.append(('assign', var_name))
            print(f"Переменная '{var_name}' используется
для присваивания, но не найдена в таблице символов.")
        else:
            operations.append(('assign', var_name))
            print(f"Переменная '{var_name}' используется
для присваивания и найдена в таблице символов.")
for pencil, token in enumerate(tokens):
    if token[0] == "NUMBER":
        for i in token[1]:
            print(global_type)
            if i not in ["0", "1", "2", "3", "4", "5", "6",
"7", "8", "9", ".", "true", "false"]:
                print(i)
                raise SyntaxError(f"Неожиданное число:
{token[1]}")
            if i == "." and global_type != "!":
                raise SyntaxError(f"Неправильный тип:
{global_type}")
print(f"Таблица символов: {symbol_table}")
return symbol_table, operations
```